University of Moratuwa

EN3150 - Pattern Recognition

Assignment 03

Resource-Constrained CNN for Edge Image Classification

Group: Draco

Dealing with AI from the core

GUNARATHNA K.T.M.B.	230203G
JAYAWARDANA K.P.P.	230294H
JITHMAL P.G.I.	230306B
NIMADITH W.N.A.C.	230442L

Repository:
`https://github.com/NavigatorPrime/CNN-Image-Classifier`

Canonical branch:
`EN3150-A03-Final-Corrected-Report-v3`

October 2026

Contents

1 Introduction and Measurement Convention **1**

2 Task 1: Data Preparation **1**

2.1 Corrected common split 1

3 Task 2: Custom Architecture Design **2**

3.1 Model A: standard CNN 2

3.2 Model B: depthwise-separable CNN 2

4 Task 3: Optimizer Selection and Tuning **3**

4.1 Controlled comparison 3

4.2 Momentum effect 4

4.3 Why Adam and learning rate 0.001 were selected 4

5 Task 4: Custom Model Training and Evaluation **4**

5.1 Model A versus Model B trade-off 5

6 Task 5: Lightweight Pre-trained Models **5**

7 Task 6: Final Comparison and Trade-off Discussion **7**

7.1 Accuracy, storage, and computation 7

7.2 Accuracy-efficiency frontier 8

7.3 Engineering recommendations 8

8 Conclusion **9**

1 Introduction and Measurement Convention

This work studies five-class DeFungi image classification under a strict custom-model parameter budget. Two custom CNNs are compared with fine-tuned MobileNetV2 and ShuffleNetV2 1.0x. The final conclusions use only the corrected leakage-free split and the clean-split Stage 2 experiment artifacts.

Every final metric in this PDF is injected from saved JSON, CSV, classification-report, training-history, checkpoint, or profiler artifacts by `scripts/generate_report_data.py`. No contaminated-split value is used. Computational cost uses one convention for an input of $1 \times 3 \times 64 \times 64$: Conv2d and Linear multiply-accumulate operations (MACs) are counted; BatchNorm, activations, pooling, residual additions, concatenation, channel shuffle, softmax, and data movement are excluded. FLOPs are reported as $2\times$ MACs. These are calculated operation counts, not device latency measurements.

2 Task 1: Data Preparation

2.1 Corrected common split

The UCI DeFungi data contain classes H1, H2, H3, H5, and H6. All packaged images are RGB and exactly 64×64 pixels. Images were grouped using both raw-file SHA-256 and decoded-pixel SHA-256 before deterministic class-aware allocation. All members of an identical-content group remain in one split.

Table 1: Corrected leakage-free dataset split.

Split	H1	H2	H3	H5	H6	Total
Train	3,083	1,634	573	573	517	6,380
Validation	661	350	123	123	111	1,368
Test	660	350	123	122	111	1,366

The split percentages are 70.00%, 15.01%, and 14.99%. The validator found 59 duplicate-content groups inside the dataset but zero raw-byte or decoded-pixel duplicate groups crossing train, validation, and test. The manifest and physical package contain the same 9,114 images.

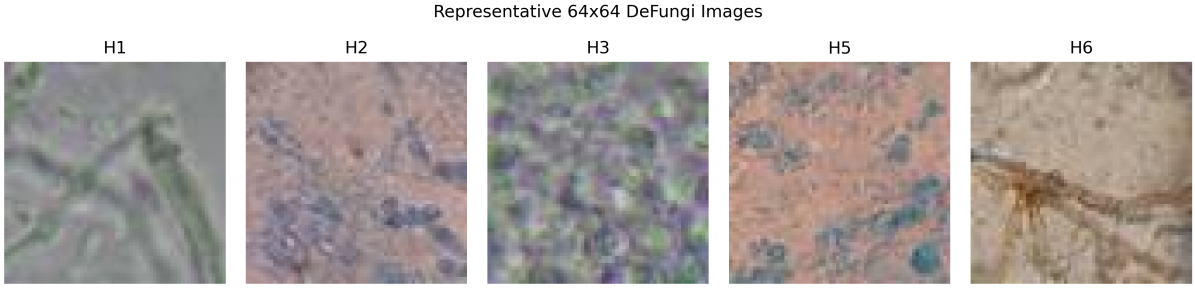


Figure 1: Representative 64×64 RGB images from the corrected training split.

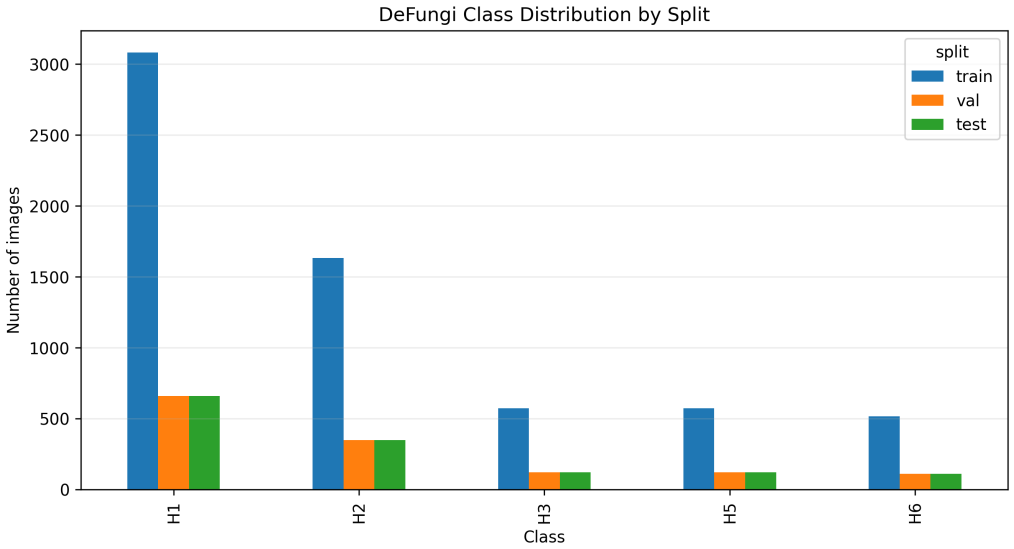


Figure 2: Per-class distribution in the corrected common split. H1 and H2 dominate, so macro-averaged metrics are reported alongside accuracy.

Training preprocessing uses resize-to-64 × 64, random horizontal and vertical flips, random rotation of ±15°, color jitter, tensor conversion, and ImageNet normalization. Validation and test preprocessing is deterministic and excludes augmentation. Seed 42 is used for reproducibility.

3 Task 2: Custom Architecture Design

3.1 Model A: standard CNN

Model A is a conventional four-block CNN. Each block uses standard 3×3 convolution, BatchNorm, ReLU, and 2×2 max pooling. The exact code-derived architecture is shown below.

Table 2: Model A architecture. ReLU, pooling, flattening, and dropout have no trainable parameters.

Stage	Layer sequence	Input	Output filters/units	Kernel	Stride/padding	Output shape	Conv/FC	BN
Block 1	Conv2D, BN, ReLU, MaxPool	$3 \times 64 \times 64$	32	3×3; pool 2×2	1/1; pool 2	$32 \times 32 \times 32$	896	64
Block 2	Conv2D, BN, ReLU, MaxPool	$32 \times 32 \times 32$	64	3×3; pool 2×2	1/1; pool 2	$64 \times 16 \times 16$	18,496	128
Block 3	Conv2D, BN, ReLU, MaxPool	$64 \times 16 \times 16$	128	3×3; pool 2×2	1/1; pool 2	$128 \times 8 \times 8$	73,856	256
Block 4	Conv2D, BN, ReLU, MaxPool	$128 \times 8 \times 8$	256	3×3; pool 2×2	1/1; pool 2	$256 \times 4 \times 4$	295,168	512
Flatten	Flatten	$256 \times 4 \times 4$	4096	—	—	4096	0	0
FC1	Linear, ReLU, Dropout(0.5)	4096	512	—	—	512	2,097,664	0
FC2	Linear, ReLU, Dropout(0.3)	512	128	—	—	128	65,664	0
Output	Linear logits	128	5	—	—	5	645	0

For a biased Conv2D layer,

$$P_{\text{conv}} = (K_h K_w C_{\text{in}} C_{\text{out}}) + C_{\text{out}},$$

and trainable BatchNorm contributes $2C_{\text{out}}$ parameters (scale and shift). For example, Block 1 has $(3{\cdot}3{\cdot}3{\cdot}32)+32 = 896$ convolution parameters and $2 \cdot 32 = 64$ BatchNorm parameters. The four blocks total 389,376 parameters. The classifier contributes

$$(4096 \cdot 512 + 512) + (512 \cdot 128 + 128) + (128 \cdot 5 + 5) = 2,163,973.$$

Therefore the verified total is $389,376 + 2,163,973 = \mathbf{2,553,349}$ trainable parameters.

Model A uses ReLU after each convolutional BatchNorm and after the first two fully connected layers. ReLU is a simple threshold operation, so it avoids the exponential evaluations used by sigmoid and tanh. This is generally favorable for hardware implementation, but no target-device latency is claimed.

3.2 Model B: depthwise-separable CNN

Model B uses one standard convolutional stem followed by six depthwise-separable blocks, global average pooling (GAP), dropout, and a linear classifier. Each block applies depthwise Conv2D, BatchNorm, ReLU6, pointwise Conv2D, BatchNorm, and ReLU6.

Table 3: Model B architecture generated from the implementation. Block parameter totals include both convolutions and both BatchNorm layers.

Stage	Input C	DW C	Groups	Kernel	Stride/pad	PW out	Output shape	Parameters	Activation
Stem	3	—	1	3×3	1/1	16	$16 \times 64 \times 64$	464	ReLU6
DS Block 1	16	16	16	3×3 DW, 1×1 PW	2/1	32	$32 \times 32 \times 32$	752	ReLU6 ×2
DS Block 2	32	32	32	3×3 DW, 1×1 PW	1/1	32	$32 \times 32 \times 32$	1,440	ReLU6 ×2
DS Block 3	32	32	32	3×3 DW, 1×1 PW	2/1	64	$64 \times 16 \times 16$	2,528	ReLU6 ×2
DS Block 4	64	64	64	3×3 DW, 1×1 PW	1/1	64	$64 \times 16 \times 16$	4,928	ReLU6 ×2
DS Block 5	64	64	64	3×3 DW, 1×1 PW	2/1	128	$128 \times 8 \times 8$	9,152	ReLU6 ×2
DS Block 6	128	128	128	3×3 DW, 1×1 PW	1/1	128	$128 \times 8 \times 8$	18,048	ReLU6 ×2
GAP	128	—	—	global	—	128	$128 \times 1 \times 1$	0	—
Dropout	128	—	—	—	—	128	128	0	—
Classifier	128	—	—	—	—	5	5 logits	645	Linear

The depthwise and pointwise convolution parameter formulas are

$$P_{\mathrm{DW}} = K_h K_w C_{\mathrm{in}} \quad \text{and} \quad P_{\mathrm{PW}} = C_{\mathrm{in}} C_{\mathrm{out}},$$

because both use `bias=False`. Their BatchNorm terms are $2C_{\mathrm{in}}$ and $2C_{\mathrm{out}}$. Thus a complete block has

$$P_{\mathrm{block}} = 9C_{\mathrm{in}} + 2C_{\mathrm{in}} + C_{\mathrm{in}} C_{\mathrm{out}} + 2C_{\mathrm{out}}.$$

For Block 1 this is $144 + 32 + 512 + 64 = 752$. For Block 2, whose output width is also 32, it is $288 + 64 + 1024 + 64 = 1,440$. The stem has $3 \cdot 16 \cdot 9 + 2 \cdot 16 = 464$ parameters and the classifier has $128 \cdot 5 + 5 = 645$. Summing every row gives:

$$\boxed{\text{Model B trainable parameters} = 37,957 < 100,000.}$$

For convolutional arithmetic alone, the depthwise-separable to standard-convolution ratio is $1/C_{\mathrm{out}} + 1/9$. It is 14.24% for the 16-to-32 and 32-to-32 blocks, and decreases for 64- and 128-output blocks. This comparison excludes BatchNorm and activation work, consistent with the project MAC convention.

Model B uses ReLU6 after the stem and after both convolution stages in every separable block. ReLU6 adds an upper clamp to ReLU's lower threshold; both remain simple piecewise-linear operations without exponential functions. The final classifier is strictly linear and produces five logits directly for `CrossEntropyLoss`; no output ReLU6 or softmax layer is applied in the model.

4 Task 3: Optimizer Selection and Tuning

4.1 Controlled comparison

The three Model B trials used the exact same corrected split, initial-state hash, seed 42, batch size 32, augmentation, cross-entropy loss, weight decay 10^{-4}, and highest-validation-accuracy checkpoint rule. Each ran for 20 epochs. All used `ReduceLROnPlateau` on validation loss with factor 0.5 and patience 4; the learning rate did not reduce during these runs.

Table 4: Controlled clean-split optimizer comparison.

Optimizer	LR	Momentum	Best val. acc.	Epoch	Final train loss	Final val. loss	Test loss	Test acc.
Standard SGD	0.010	0.0	66.01%	18	0.8961	0.8384	0.8358	65.67%
SGD + Momentum	0.010	0.9	70.54%	18	0.7938	0.7165	0.7255	68.81%
Adam	0.001	–	**73.25%**	20	**0.7510**	**0.6671**	**0.6755**	**70.57%**

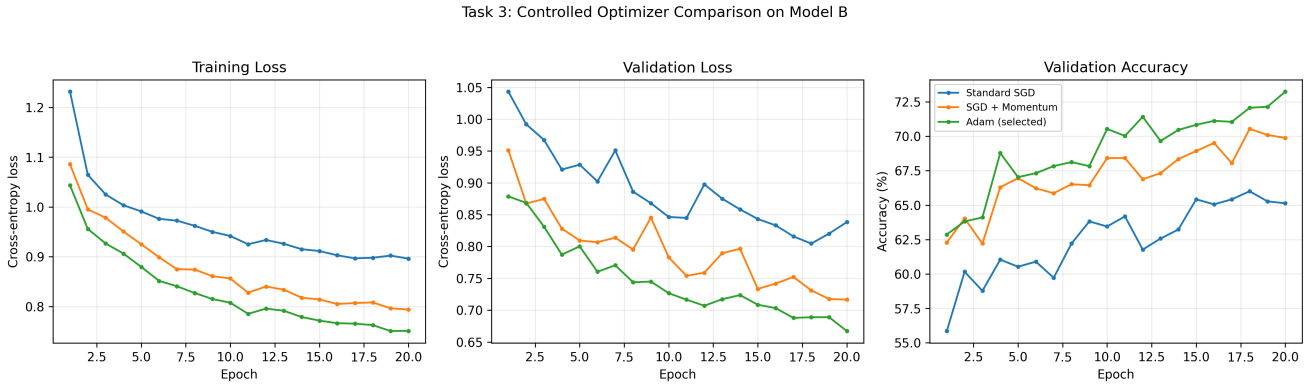


Figure 3: Training loss, validation loss, and validation accuracy for the controlled 20-epoch optimizer comparison.

4.2 Momentum effect

Momentum accumulates a velocity in directions supported by successive gradients:

$$v_t = \mu v_{t-1} + g_t, \qquad \theta_{t+1} = \theta_t - \eta v_t.$$

This can reduce oscillation and accelerate progress along consistent directions. The corrected curves support that interpretation: by epoch 10, SGD with momentum reached 68.42% validation accuracy compared with 63.45% for plain SGD. Momentum also achieved a higher best validation accuracy (70.54% versus 66.01%), lower final validation loss (0.7165 versus 0.8384), and higher test accuracy (68.81% versus 65.67%).

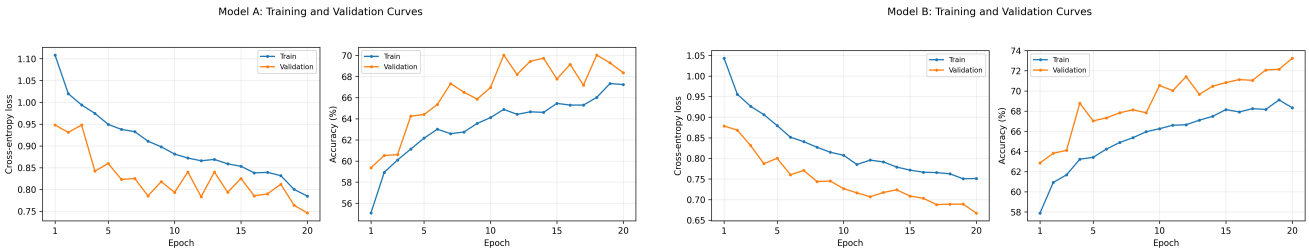
4.3 Why Adam and learning rate 0.001 were selected

Adam was selected because it produced the strongest controlled result within the fixed 20-epoch budget: 73.25% best validation accuracy, 70.57% test accuracy, and macro F1 0.6800. These exceed both plain SGD and SGD with momentum. Its curves also show useful progress throughout training, reaching 70.54% validation accuracy by epoch 10 and the best checkpoint at epoch 20. Adam combines momentum-like first-moment estimation with second-moment scaling, adapting the effective step size for each parameter. This reduces the amount of manual gradient-scale tuning required by plain SGD, while the conclusion here remains grounded in the measured comparison rather than optimizer theory alone.

The base learning rate 0.001 is a standard practical starting point for Adam and is smaller than the 0.01 used for the SGD variants. Because Adam rescales updates with first- and second-moment estimates, this smaller base rate helps avoid excessively large parameter updates while still permitting progress. In the corrected experiment it produced stable decreasing losses and the best validation result within 20 epochs. The plateau scheduler was configured but did not need to lower the rate, so all recorded Adam epochs used 0.001.

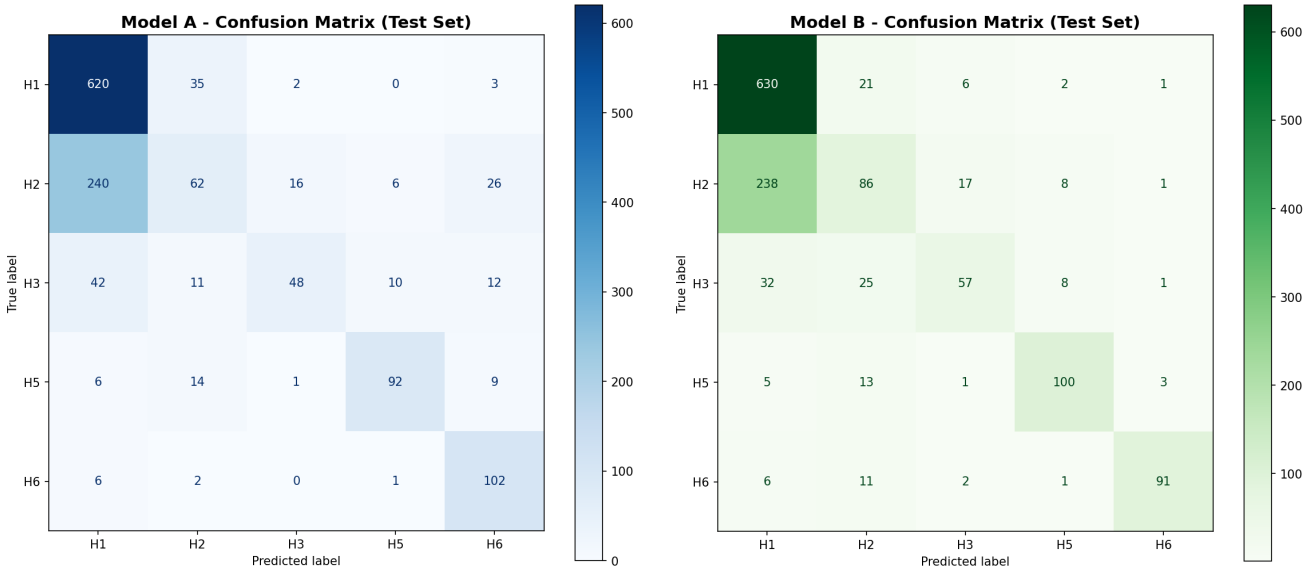
5 Task 4: Custom Model Training and Evaluation

Both canonical custom-model runs used the corrected split for 20 epochs and evaluated the test set only after restoring the selected checkpoint.



(a) Model A. **(b)** Model B.

Figure 4: Training and validation loss and accuracy for the two custom models.



(a) Model A.

(b) Model B.

Figure 5: Corrected test-set confusion matrices; each totals 1,366 images.

Table 5: Canonical clean-split custom-model results. Checkpoint sizes use binary MiB/KiB.

Model	Params	Size	Test loss	Test acc.	Macro P	Macro R	Macro F1	Total time	Time/ep	MACs	FLOPs
Model A	2,553,349	9.755 MiB	0.8360	67.64%	0.6820	0.6360	0.6254	346.43 s	17.32 s	62.325 M	124.651 M
Model B	37,957	179.8 KiB	0.6755	70.57%	0.7416	0.6606	0.6800	253.80 s	12.69 s	7.263 M	14.526 M

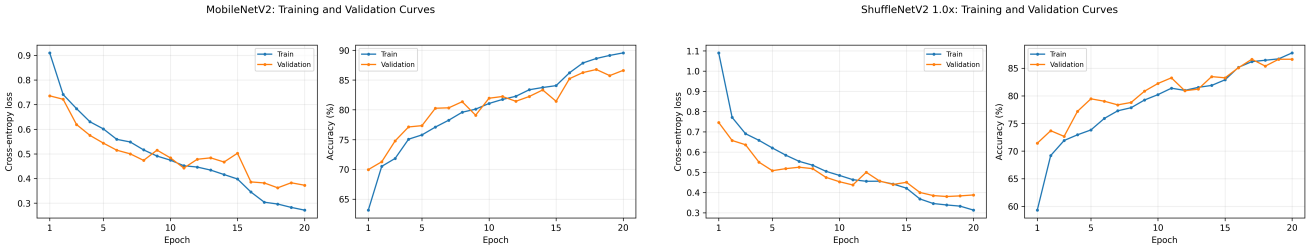
5.1 Model A versus Model B trade-off

Model B achieved 2.93 percentage points higher test accuracy and a higher macro F1 (0.6800 versus 0.6254) despite being dramatically smaller. It uses 98.51% fewer parameters, or 67.27× fewer, and 88.35% fewer calculated MACs. Its checkpoint is 179.8 KiB compared with 9.755 MiB for Model A, and its average training epoch was shorter in this environment.

Model A remains a useful conventional baseline: standard convolutions and a large fully connected head offer greater nominal representational capacity and are straightforward to explain. In this experiment, however, that capacity did not produce higher clean-split accuracy. Model B's depthwise-separable blocks, global pooling, much smaller storage footprint, and lower arithmetic count make it more attractive for resource-constrained exploration. This does not prove deployment feasibility: checkpoint storage, parameter count, calculated MACs, runtime activation RAM, and measured inference latency are different quantities. Peak RAM and device latency were not measured.

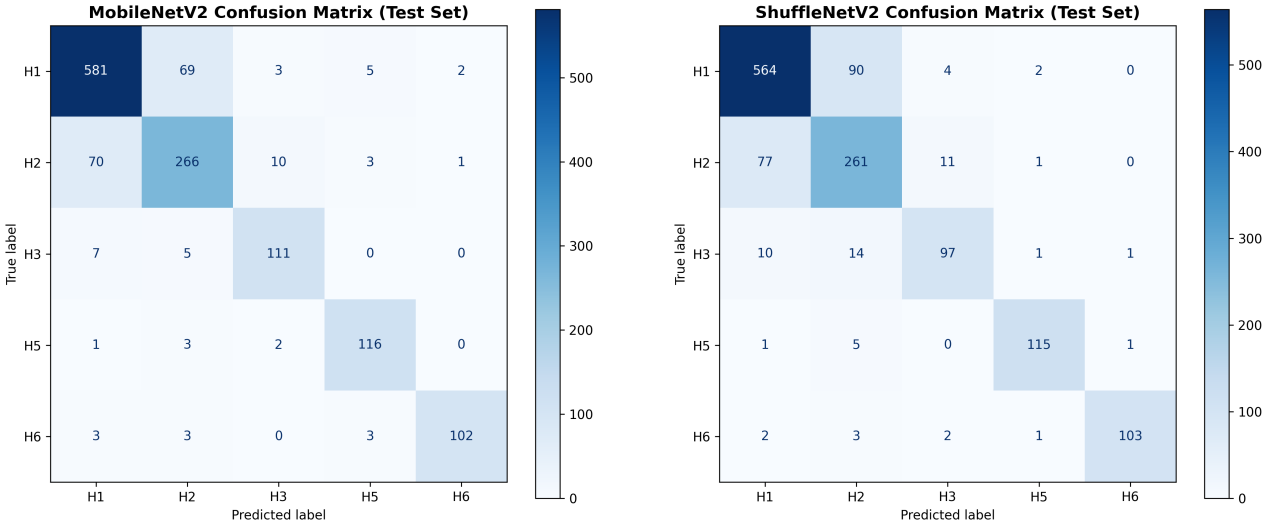
6 Task 5: Lightweight Pre-trained Models

MobileNetV2 used `torchvision.models.MobileNet_V2_Weights.DEFAULT`; ShuffleNetV2 1.0x used `ShuffleNet_V2_X1_0_Weights.DEFAULT`. Each ImageNet classifier was replaced with a five-class Linear layer and all parameters were fine-tuned on the same corrected split for 20 epochs. Training used Adam with base learning rate 5×10^{-4}, weight decay 10^{-4}, and `ReduceLROnPlateau` on validation loss. The lowest-validation-loss checkpoint was evaluated.



(a) MobileNetV2. **(b)** ShuffleNetV2 1.0x.

Figure 6: Fine-tuning and validation curves for the two pre-trained models.



(a) MobileNetV2. **(b)** ShuffleNetV2 1.0x.

Figure 7: Corrected test-set confusion matrices for the fine-tuned models.

Table 6: Canonical clean-split Task 5 results.

Model	Params	Size	Loss	Accuracy	Macro P	Macro R	Macro F1	Total time	Time/ep	MACs/FLOPs
MobileNetV2	2,230,277	8.740 MiB	0.3581	86.09%	0.8824	0.8825	0.8822	753.41 s	37.66 s	24.455/48.910 M
ShuffleNetV2 1.0x	1,258,729	4.966 MiB	0.3968	83.46%	0.8705	0.8519	0.8606	482.69 s	24.13 s	11.751/23.502 M

MobileNetV2 gives the highest accuracy and macro F1. ShuffleNetV2 is smaller and uses fewer MACs while still substantially outperforming Model B. Pre-training likely contributes reusable visual representations, but the advantage cannot be assigned to pre-training alone because architecture and capacity also differ.

7 Task 6: Final Comparison and Trade-off Discussion

7.1 Accuracy, storage, and computation

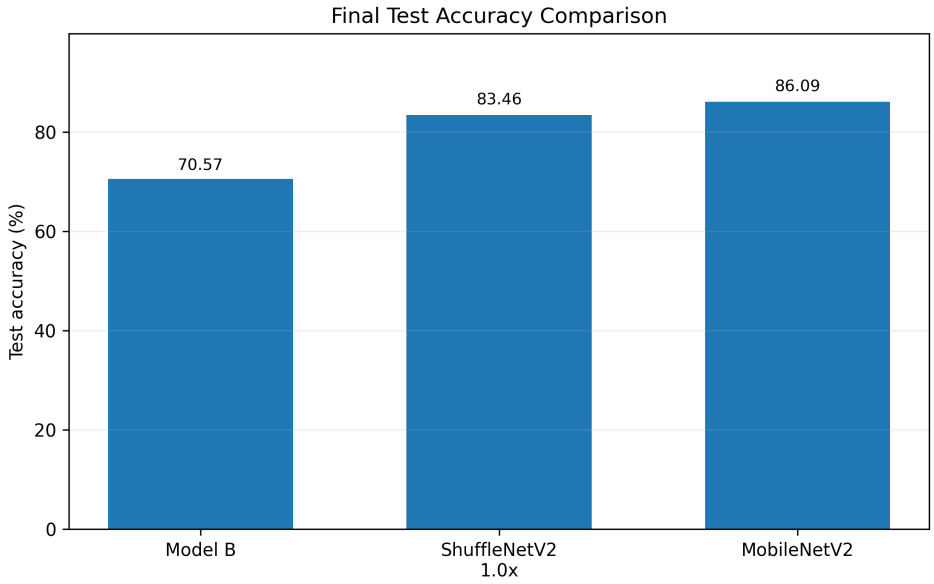
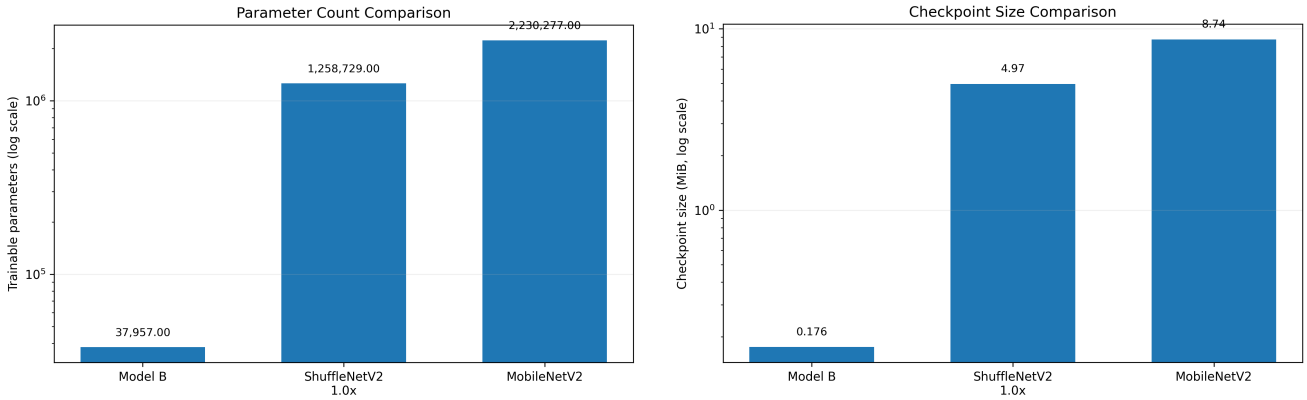


Figure 8: Corrected test accuracy for Model B and the two fine-tuned pre-trained models.

MobileNetV2 improves on Model B by 15.52 percentage points; ShuffleNetV2 improves by 12.88 points. Their H2 recalls are 0.7600 and 0.7457, compared with 0.2457 for Model B.



(a) Trainable parameters.

(b) Checkpoint storage in MiB.

Figure 9: Parameter and actual checkpoint-size comparisons. Log scales expose the large ratios.

Relative to Model B, MobileNetV2 has 58.76× the parameters and a 49.79× larger checkpoint. ShuffleNetV2 has 33.16× the parameters and a 28.29× larger checkpoint.

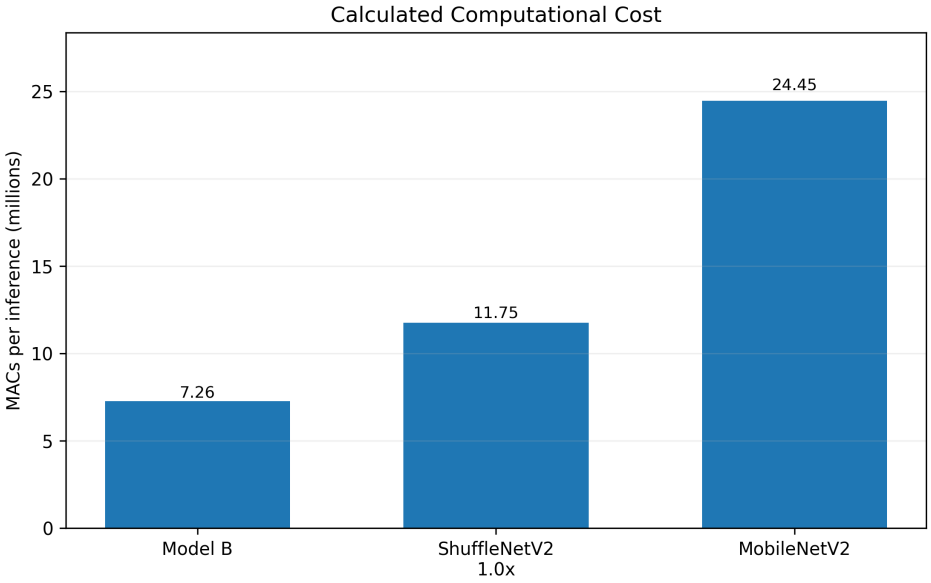
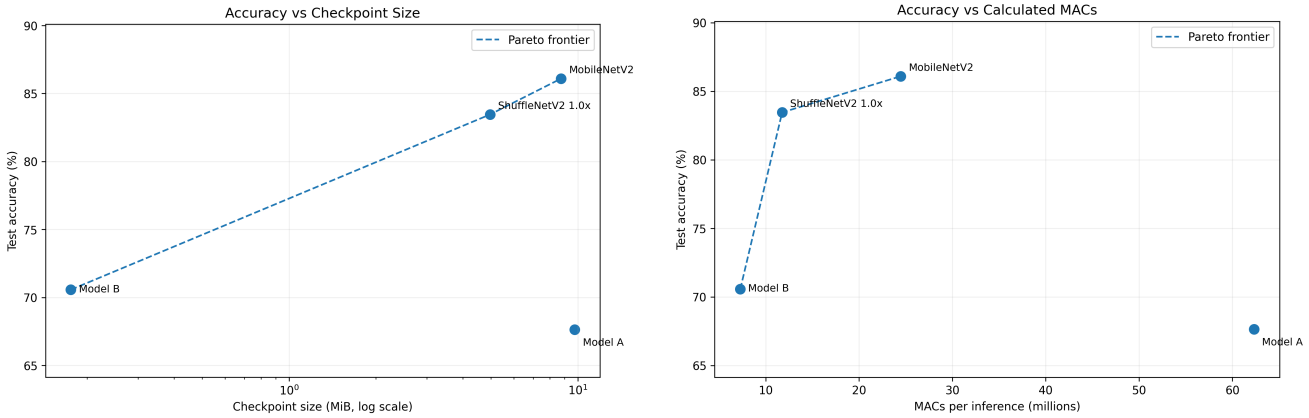


Figure 10: Calculated Conv2d/Linear MACs per 64×64 image.

MobileNetV2 uses 3.37× Model B's MAC count and ShuffleNetV2 uses 1.62×. Model B is therefore the lowest-cost custom option, whereas the pre-trained models combine larger capacity, different efficient building blocks, and learned ImageNet representations to obtain higher accuracy.

7.2 Accuracy-efficiency frontier



(a) Accuracy versus checkpoint size.

(b) Accuracy versus MACs.

Figure 11: Corrected accuracy-efficiency trade-offs. Model A is dominated by Model B; the remaining three models provide distinct resource/accuracy operating points.

7.3 Engineering recommendations

- **Strictest parameter, storage, or arithmetic budget: Model B.** It is trained from scratch, contains only 37,957 parameters, and has the smallest checkpoint and MAC count. Quantization, peak-RAM profiling, operator support, and on-device latency must still be measured.
- **Balanced accuracy and resource cost: ShuffleNetV2 1.0x.** It reaches 83.46% accuracy with substantially fewer parameters, checkpoint storage, and MACs than MobileNetV2.
- **Accuracy-first choice: MobileNetV2.** It reaches 86.09% accuracy and macro F1 0.8822, the strongest measured classification result, with the highest compute cost among the three final candidates.

8 Conclusion

On the leakage-free split, Model B outperforms the much larger conventional Model A while using 98.51% fewer parameters and 88.35% fewer MACs. MobileNetV2 and ShuffleNetV2 produce considerably higher classification performance, but their advantage accompanies greater capacity, different architectures, and ImageNet pre-training. The correct engineering choice depends on the real storage, RAM, latency, energy, and accuracy constraints; this work measures checkpoint storage and training time and calculates MACs, but does not claim unmeasured MCU RAM or latency.

Reproducibility and Submission Reminder

All seven experiments use the same package SHA-256 and manifest SHA-256 recorded in their JSON metadata. Every history contains 20 epochs, every confusion matrix totals 1,366, and cross-split duplicate content equals zero. Canonical artifacts are under `results_clean_split/`; the older `results/` tree is historical only.

The assignment's dataset information sheet is external to this GitHub repository. The group must update and submit that external sheet separately; this report does not claim it has been completed.

References

[1] UCI Machine Learning Repository, *DeFungi*, Dataset 773. `https://archive.ics.uci.edu/dataset/773/defungi`.

[2] M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," *CVPR*, 2018.

[3] N. Ma et al., "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," *ECCV*, 2018.

[4] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," *ICLR*, 2015.

[5] PyTorch and Torchvision model implementations, `https://pytorch.org/vision/stable/models.html`.